

Expresiones y operadores booleanos

Luisa Gomez

luisa.gomez@pucp.edu.pe

<https://github.com/4591526>

Expresiones booleanas

- Los booleanos son un tipo de dato que permite almacenar dos valores **True** o **False**.
- Un valor booleano también puede ser el resultado de evaluar una expresión.
- Las expresiones booleanas son las estructuras fundamentales de la programación.
- ¡Recuerda que el signo **'=**' se reserva para asignación! Las expresiones booleanas observan variables pero nunca las cambian.

Operadores comparativos

	Operador	Descripción	Ejemplo
0	==	Igual a	5 == 5
1	!=	Distinto de	5 != 3
2	>	Mayor que	7 > 3
3	<	Menor que	3 < 7
4	>=	Mayor o igual que	3 >= 5
5	<=	Menor o igual que	3 <= 5

Operadores de pertenencia

	Operador	Descripción	Ejemplo
0	in	Verifica si un elemento está presente en una lista o cadena	'a' in 'manzana'
1	not in	Verifica si un elemento no está presente en una lista o cadena	'b' not in 'manzana'

Operadores lógicos

	Operador	Descripción	Ejemplo
0	and	Devuelve True si ambas expresiones son verdaderas	(5 > 3) and (7 > 5)
1	or	Devuelve True si al menos una expresión es verdadera	(5 > 3) or (7 < 5)
2	not	Devuelve True si la expresión es falsa	not(5 > 3)



input() en Python

Permite recibir datos del usuario desde el teclado.



? ¿Qué hace?

`input()` muestra un mensaje (opcional) en la pantalla y espera que el usuario escriba algo y presione Enter.

Lo que el usuario escribe se almacena como una cadena de texto (`str`).

input(mensaje)

Función input()

Función integrada de Python que espera una entrada del usuario.

Mensaje (opcional)

Texto que se muestra al usuario como una invitación o pregunta. Puede omitirse.

Entrada del usuario (str)

El valor que escribe el usuario y presiona Enter. Siempre se recibe como cadena de texto (`str`).

💡 ¿Para qué sirve?

- Interactuar con el usuario.
- Pedir información como nombre, edad, opciones, etc.
- Crear programas más dinámicos e interactivos.
- Tomar decisiones según lo que escriba el usuario.



Métodos de str en Python



Funciones (métodos) útiles para trabajar con cadenas de texto (strings).



¿Qué es un método de str?

Es una función que se aplica a un objeto de tipo cadena (str) para realizar una acción y devolver un resultado.

`texto.método()`

↓
cadena
(str)

↓
método



Sintaxis general

`cadena.método(argumentos)`

↓
la cadena
(str)

↓
método a
usar

↓
argumentos
(opcionales)



Importante

Los métodos no modifican la cadena original (son inmutables). Devuelven una nueva cadena o un valor.

```
texto = "Hola"
nuevo = texto.upper() # devuelve 'HOLA'
# 'texto' sigue siendo 'Hola'
```

Métodos de str más utilizados

	Método	¿Qué hace?	Sintaxis	Ejemplo	Resultado
	<code>upper()</code>	Convierte todos los caracteres a mayúsculas.	<code>texto.upper()</code>	<code>"hola mundo".upper()</code>	'HOLA MUNDO'
	<code>lower()</code>	Convierte todos los caracteres a minúsculas.	<code>texto.lower()</code>	<code>"HOLA MUNDO".lower()</code>	'hola mundo'
	<code>startswith(pat)</code>	Verifica si la cadena empieza con el texto dado.	<code>texto.startswith(pat)</code>	<code>"Python es genial".startswith("Py")</code>	True
	<code>endswith(pat)</code>	Verifica si la cadena termina con el texto dado.	<code>texto.endswith(pat)</code>	<code>"archivo.txt".endswith(".txt")</code>	True
	<code>count(sub)</code>	Cuenta cuántas veces aparece el texto dado.	<code>texto.count(sub)</code>	<code>"banana".count("a")</code>	3
	<code>find(sub)</code>	Devuelve la posición (índice) de la primera aparición. Si no existe, -1.	<code>texto.find(sub)</code>	<code>"banana".find("na")</code>	2
	<code>index(sub)</code>	Devuelve la posición (índice) de la primera aparición. Si no existe, da error.	<code>texto.index(sub)</code>	<code>"banana".index("na")</code>	2
	<code>replace(ant, nuev)</code>	Reemplaza todas las apariciones del texto 'ant' por 'nuev'.	<code>texto.replace(ant, nuev)</code>	<code>"gato negro".replace("negro", "blanco")</code>	'gato blanco'
	<code>split(sep)</code>	Divide la cadena usando el separador indicado y devuelve una lista.	<code>texto.split(sep)</code>	<code>"a,b,c".split(",")</code>	['a', 'b', 'c']
	<code>len()</code>	Devuelve la longitud (número de caracteres) de la cadena.	<code>len(texto)</code>	<code>len("Hola mundo")</code>	10